DEVELOPER ONBOARDING GUIDE

GCMS Developer Onboarding & Run Guide

Complete Setup, Download, Configuration, Seeding & Execution Manual

Project:	GCMS Clinical AI Consultation Backend
Document Type:	New Developer Setup & Run Guide
Target Audience:	Backend Engineers, DevOps, Full-Stack Developers
Prerequisites:	Python 3.11+, Docker Desktop / Colima, Ollama
Repository:	https://github.com/HponeMyatThu/Clinic-ai.git
Publication Date:	November 2026

CONFIDENTIAL ? For Authorized Healthcare, Clinical & Engineering Stakeholders Only
Complies with Singapore MOH, HSA, and PDPA Healthcare Guidelines

1. Prerequisites & Environment Requirements

Welcome to the GCMS Clinical AI Backend project. This guide covers everything needed to clone, configure, seed, and run the backend locally on macOS (Apple Silicon recommended) or Linux.

* Operating System ? macOS (Apple Silicon M1/M2/M3/M4 recommended for Metal acceleration) or Ubuntu 22.04+.
* Python Runtime ? Python 3.11 or Python 3.12 (Python 3.12 recommended; macOS command: brew install python@3.12).
* Container Engine ? Docker & Docker Compose (Docker Desktop or Colima for PostgreSQL and Redis).
* Local AI Runtime ? Ollama installed natively on the host (command: brew install ollama).

2. Step-by-Step Installation & Setup

Step 2.1: Clone and Create Virtual Environment

```
git clone https://github.com/HponeMyatThu/Clinic-ai.git gcms_ai_backend
cd gcms_ai_backend
python3.12 -m venv .venv
source .venv/bin/activate
```

Step 2.2: Install Project Dependencies

```
pip install -r requirements.txt
```

Step 2.3: Configure Environment Variables

Copy the template configuration file to .env:

```
cp .env.example .env
```

3. Starting Database & Local AI Services

Step 3.1: Start PostgreSQL and Redis via Docker

```
docker compose up -d
# Verify containers are running healthy:
docker compose ps
```

Step 3.2: Start Ollama and Pull the Clinical Model

Run Ollama natively on macOS to take full advantage of Apple Silicon unified memory and Metal GPU acceleration:

```
# Start Ollama daemon in a terminal:
ollama serve

# In a separate terminal, pull the recommended 8B reasoning model:
ollama pull qwen3:8b
# (Alternative option: ollama pull llama3.1:8b)
```

4. Database Seeding & Schema Verification

The project comes with a synthetic development dataset containing 50 primary care SNOMED CT concepts and 40 Singapore Drug Dictionary (SDD) medications:

```
# Run the synthetic seed script inside (.venv):
python -m scripts.seed_dev_data
```

> **[SUCCESS] Verifying Database in Docker**
> docker exec -it gcms-postgres psql -U gcms -d gcms_ai -c 'SELECT count(*) FROM snomed_concepts;' Should return 50 rows for concepts and 196 rows for descriptions.

5. Running the Application & Testing APIs

Step 5.1: Launch the FastAPI Application

```
uvicorn app.main:app --reload --port 8000
```

Step 5.2: Access Interactive API Documentation

* Swagger UI ? http://localhost:8000/docs
* ReDoc Interface ? http://localhost:8000/redoc
* Health & Readiness Probe ? http://localhost:8000/api/v1/health

Step 5.3: Example Test Commands

```
# 1. Test SNOMED Search:
curl 'http://127.0.0.1:8000/api/v1/terminology/snomed/search?q=asthma'

# 2. Test SDD Medication Search:
curl 'http://127.0.0.1:8000/api/v1/terminology/sdd/search?q=panadol'
```

6. Troubleshooting Guide

* Error: relation 'snomed_descriptions' does not exist ? Run 'python -m scripts.seed_dev_data' to create and populate the database tables.
* Error: Connection refused on port 5432 ? Ensure PostgreSQL is running: 'docker compose up -d'.
* Ollama unreachable in /health ? Ensure Ollama is running on localhost:11434 with 'ollama serve'.
* ModuleNotFoundError ? Ensure you activated the virtual environment: 'source .venv/bin/activate'.